

用語集

競技プログラミング基本用語集

[解説動画はこちら](#)です。

1. 概要

本講座では、競技プログラミングの問題文や解説、AtCoder Algorithm Lectures で頻出の用語や記号のうち、特に多くの分野に共通するものを簡単にまとめます。

特定の分野やアルゴリズムに特有の用語や記号については、基本的には個別の講座内で定義されるため、本講座では解説を省略しています。またいくつかの分野では、分野ごとの用語集も用意しているので、それらも参照してください。

- [グラフ理論用語集](#)
- [代数構造用語集](#)

用語について、日本の中学・高校で通常扱うものについては、省略されている場合があります。逆に中学・高校で通常学ぶものであっても、競技プログラミングの問題文や解説（国内、国外）で少し違う意味や記号で登場する可能性があるものは、意図的に扱っている場合があります。

2. 前提知識

特にありません。

3. 基本的な数や記号

3.1. 不等号

a が b 以下であることを, $a \leq b$ や $a \leq b$ と書きます. a が b 以上であることを, $a \geq b$ や $a \geq b$ と書きます.

日本の高校までの教育では $\leq, \geq$ がよく用いられていますが, 大学での学習や競技プログラミングの文献では $\leq, \geq$ もよく用いられています. AtCoder Algorithm Lectures でも $\leq, \geq$ を用います.

3.2. 非負整数, 正整数

0 以上の整数 (負でない整数) を**非負整数** (non-negative integer), 0 より大きい整数を**正整数** (positive integer) といいます.

「自然数」という用語もありますが, 非負整数と同じ意味で使われる場合も, 正整数と同じ意味で使われる場合もあるため, AtCoder Algorithm Lectures 内では使用しません.

同様に, 0 以上の実数を**非負実数**, 0 より大きい実数を**正実数**といいます.

3.3. 床関数

実数 x に対して, x 以下の整数のうち最大のものを $\lfloor x \rfloor$ と書きます. x に $\lfloor x \rfloor$ を対応させる関数を**床関数** (floor function) といいます.

x	$\lfloor x \rfloor$
-1	-1
0	0
1	1
1.23	1
-(1.23)	-2

これは「 x の小数部分の切り捨て」「 x の整数部分」と考えてもらえると理解しやすいと思いますが, $x < 0$ の場合に誤解が生じる可能性があるので注意してください.

実数 x に対して、 $\lfloor x \rfloor \leq x < \lfloor x \rfloor + 1$ が成り立ちます。

3.4. 天井関数

実数 x に対して、 x 以上の整数のうち最小のものを $\lceil x \rceil$ と書きます。 x に $\lceil x \rceil$ を対応させる関数を**天井関数**（ceil function）といいます。

x	$\lceil x \rceil$
-1	-1
0	0
1	1
1.23	2
-(1.23)	-1

これは「 x の小数部分の切り上げ」と考えてもらえると理解しやすいと思いますが、 $x < 0$ の場合に誤解が生じる可能性があるので注意してください。

実数 x に対して、 $\lceil x \rceil - 1 < x \leq \lceil x \rceil$ が成り立ちます。

4. 集合と写像

4.1. 集合

いくつかの互いに異なるものからなる集まりを**集合**（set）といいます。（これ以上の形式的な定義はせず、ここでは直感的な説明にとどめます。）

S を集合とすると、「もの」 x に対して、 x が S に含まれるか否かが定まります。 x が S に含まれる場合には $x \in S$ ，そうでない場合には $x \notin S$ と書きます。 $x \in S$ であるとき x を S の**要素**（element），あるいは S の**元**といいます。

集合を表す際には、次のような記号があります。

- 外延的記法：例えば $\{2, 5, 10\}$ は、 $2, 5, 10$ の 3 つの要素からなる集合です。 集合では（列と違って）要素を並べる順を区別せず、例えば $\{2, 5, 10\}$ と $\{10, 2, 5\}$ は同じ集合

です.

- 内包的記法：例えば $\{x \mid x \text{ は整数}\}$ は、すべての整数からなる集合です. 縦線 $|$ の左側に集合に含めるもの、右側に含めるための条件を書くという記法です.

特別重要な集合には、固有の記号を使うことがあります. ここでは特に、次の記号を導入します.

- $\emptyset$: **空集合**. 要素を持たない集合.
- $\mathbb{R} = \{x \mid x \text{ は実数}\}$: すべての実数からなる集合.
- $\mathbb{Z} = \{x \mid x \text{ は整数}\}$: すべての整数からなる集合.
- $\mathbb{Z}_{\geq 0} = \{x \in \mathbb{Z} \mid x \geq 0\}$: すべての非負整数からなる集合.

4.2. 多重集合

集合 S といった場合には、要素は互いに異なることが前提で、あるもの x について $x \in S$ であるか否かの区別が重視されます.

一方で、同じ要素が複数あることを許し、 $x \in S$ であるか否かに加えて x がいくつ含まれているかという情報も含めて考えたものを**多重集合** (multiset) といいます.

例えば 1 を 3 つ、2 を 4 つ含む多重集合は

$$\{1, 1, 1, 2, 2, 2, 2\}$$

と表記します. 文献によっては集合との区別を明示して、波括弧を二重にした $\{\{1, 1, 1, 2, 2, 2, 2\}\}$ などの表記をすることもあります. AtCoder Algorithm Lectures では、多重集合と断った上で、集合と同様の表記を用います.

4.3. 要素数

要素が有限個であるような集合を**有限集合** (finite set), そうでない集合を**無限集合** (infinite set) といいます.

有限集合 S に対して、 S の要素数を、 $|S|$ や $\#S$ などと表します. 同じ記号を、多重集合の要素数、配列の長さ、文字列の長さなどにも用います. 無限集合について、 $|S| = \infty$ と書くこともあります.

要素数が非負整数 n に等しい集合を **n 元集合**といいます.

4.4. 部分集合

A, B を集合とします. $x \in A \implies x \in B$ が成り立つとき, A は B の**部分集合** (subset) であるといい, $A \subseteq B$ や $A \subset B$ と書きます.

$A \subseteq B$ かつ $B \subseteq A$ であるとき, A と B は**等しい**といい, $A = B$ と書きます.

$A \subseteq B$ かつ $A \neq B$ であるとき, A は B の**真部分集合** (proper subset) であるといい, $A \subsetneq B$ と書きます.

文献によっては $A \subset B$ と書いた場合に, $A \subsetneq B$ を意味する場合があります. AtCoder Algorithm Lectures では, $\subset$ は使わずに $\subseteq, \subsetneq$ を使う予定です.

n を非負整数とし, S を n 元集合とすると, S の部分集合は 2^n 個あります.

4.5. 二項係数

n を非負整数とし, S を n 元集合とすると, S の k 元部分集合の個数を $\binom{n}{k}$ と書き, **二項係数** (binomial coefficient) といいます. 特に, $k < 0$ や $n < k$ のときには $\binom{n}{k} = 0$ であると定義します.

二項係数について, ${}_nC_k, C(n, k)$ などの記号が用いられる文献もあります.

4.6. 集合演算

A, B を集合とすると, これらをもとに次のような集合を考えることがあります.

- A と B の**和集合** (union) $A \cup B : \{x \mid x \in A \text{ または } x \in B\}$.
- A と B の**差集合** (set difference) $A \setminus B : \{x \mid x \in A \text{ かつ } x \notin B\}$.
- A と B の**共通部分** (intersection) $A \cap B : \{x \mid x \in A \text{ かつ } x \in B\}$.

なお, 共通部分については, 積集合と呼ぶ文献もありますが, 直積集合と呼ばれる別の概念との混同の可能性もあるため, AtCoder Algorithm Lectures では採用しません.

ある集合が A, B の和集合であり、 A, B の共通部分が空集合である場合には、**非交和** (disjoint union) といい、 $A \amalg B$ や $A \sqcup B$ と書きます。

4.7. 写像

集合 A, B に対して、 A の各要素 x に B の要素 $f(x)$ を 1 つ対応させる規則 f を**写像** (map) または**関数** (function) といい、 $f: A \rightarrow B$ と書きます。

A, B が有限集合で、 $|A| = n, |B| = m$ であるとき、 A から B への写像は m^n 個あります。(なお、AtCoder Algorithm Lectures では $0^0 = 1$ と約束します。)

4.8. 単射，全射，全単射

写像 $f: A \rightarrow B$ について、次を定義します。

- f が**単射** (injective) であるとは、 $x \neq y$ ならば $f(x) \neq f(y)$ が成り立つことです。
- f が**全射** (surjective) であるとは、任意の $b \in B$ に対して $f(x) = b$ を満たす $x \in A$ が存在することです。
- f が単射かつ全射であるとき、 f は**全単射** (bijective) であるといいます。

全単射は A と B の要素の **1 対 1 の対応** (one to one correspondence) といいます。

A, B が有限集合のとき、 $f: A \rightarrow B$ が単射ならば $|A| \leq |B|$ ，全射ならば $|A| \geq |B|$ ，全単射ならば $|A| = |B|$ が成り立ちます。

4.9. 最小値・最大値

S を実数からなる集合（あるいは要素の大小比較ができる集合）とします。 x が S の**最小値** (minimum) であるとは

- $x \in S$ であり、
- 任意の $y \in S$ に対して $x \leq y$ が成り立つ

ことをいいます。 $\{2, 5, 10\}$ の最小値は 2 です。

どのような集合にも最小値があるわけではありません。例えば

- $\emptyset$ には最小値は存在しません。
- $\mathbb{Z}$ には最小値は存在しません。
- 正実数全体には最小値が存在しません。

空でない有限集合には最小値が存在します。

S の最小値を $\min S$ と書きます。多重集合や列についても同様の記号を用います。

$\{a_1, a_2, \dots, a_n\}$ の最小値を $\min(a_1, a_2, \dots, a_n)$ と書きます。例えば

$$\min(a, b) = \begin{cases} a & (a \leq b) \\ b & (a > b) \end{cases}$$

です。この値は「 a と b の小さい方」あるいは $a = b$ の場合の正確性のために「 a と b の大きくない方」と説明されることもあります。

最大値 (maximum) についても同様に定義され、 $\min$ の代わりに $\max$ という記号が用いられます。

4.10. mex

非負整数からなる有限集合 S について、 S に含まれない最小の非負整数を S の **mex**

(minimum excluded value, 最小除外数) といい、 $\text{mex } S$ と書きます。多重集合や列についても同様の記号を用います。例えば、以下が成り立ちます。

- $\text{mex } \emptyset = 0$.
- $\text{mex } \{0, 1, 2, 4, 5, 7\} = 3$.
- $\text{mex } \{1, 2, 4, 5, 7\} = 0$.

問題によっては、正整数からなる有限集合 S について、 S に含まれない最小の正整数を $\text{mex } S$ としているものもあります。

5. 総和，総積

5.1. 総和記号 $\sum$

整数 l, r ($l \leq r$) について,

$$\sum_{i=l}^r a_i = a_l + a_{l+1} + \cdots + a_r$$

と書きます. 右辺 $a_l + a_{l+1} + \cdots + a_r$ は表記上 3 つ以上の数を足しているように見えるため, $r = l, l+1$ の場合などに違和感を感じるかもしれませんが, $l \leq i \leq r$ であるような i に対する総和と解釈することが期待された表記として一般的です.

この値を, i に関する条件を $\sum$ 記号の下に書くという形で

$$\sum_{l \leq i \leq r} a_i$$

のように表すこともあります. 他に集合 S に対して, S の要素の総和を

$$\sum_{x \in S} x$$

とも書くなど, いくつかのバリエーションがあります.

足す対象の数がひとつもない場合の総和, 例えば

- $l = r$ のときの $\sum_{l \leq i < r} a_i$
- $S = \emptyset$ のときの $\sum_{x \in S} x$

などは 0 と定義します. これと同じ理由で, $r = l - 1$ の場合に $\sum_{i=l}^r a_i = 0$ とすることもあります.

5.2. 総積記号 $\prod$

$\sum$ は対象となる数の総和を表す記号ですが, 同じように対象となる数の総積を表す記号として, $\prod$ が用いられます. 例えば

$$\prod_{i=2}^5 (2i+1) = 5 \times 7 \times 9 \times 11$$

です。記号の下に条件を書く $\prod_{x \in S} x$ のような表記も $\sum$ と同様に用います。

掛ける対象の数がひとつもない場合の総積（**空積**，empty product）は，1 であると定義します。例えば， $S = \emptyset$ のとき

$$\prod_{x \in S} x = 1$$

です。

5.3. 階乗

非負整数 n に対して

$$n! = \prod_{i=1}^n i$$

を n の**階乗**（factorial）といいます。特に空積の約束により $0! = 1$ です。

$$0! = 1, \quad 1! = 1, \quad 2! = 2, \quad 3! = 6, \quad 4! = 24, \quad 5! = 120.$$

互いに異なる n 個のものをすべて 1 回ずつ用いて並べた列を**順列**（permutation）といいます。順列の総数は $n!$ です。

また， $0 \leq k \leq n$ のとき，二項係数について $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ が成り立ちます。

6. 列と添字

6.1. 0-based index, 1-based index

配列の最初の要素の添字を 0 とする方法を 0-based index (0-indexed)，1 とする方法を 1-based index (1-indexed) といいます。

6.2. 区間

l, r を $l \leq r$ を満たす実数とすると、次の形の集合のうち空でないものを**区間** (interval) といいます。

- $[l, r] = \{x \in \mathbb{R} \mid l \leq x \leq r\}$.
- $[l, r) = \{x \in \mathbb{R} \mid l \leq x < r\}$.
- $(l, r] = \{x \in \mathbb{R} \mid l < x \leq r\}$.
- $(l, r) = \{x \in \mathbb{R} \mid l < x < r\}$.

$[l, r]$ を**閉区間** (closed interval), (l, r) を**开区間** (open interval), $[l, r)$ を (左閉, 右開の) **半开区間** (half open interval) といいます。

$l = -\infty$ とした

- $(-\infty, r] = \{x \in \mathbb{R} \mid x \leq r\}$
- $(-\infty, r) = \{x \in \mathbb{R} \mid x < r\}$

も区間といいます ($r = \infty$ も同様です)。

空集合を区間として扱うかには定義揺れがあります。

これらを、整数に限定した区間 (**整数区間**, integer interval) も、混乱の恐れが少ない場合には同じ記号で表記します。特に配列の添字集合について使われることが多いです。この場合、例えば

$$\begin{aligned}[2, 5] &= \{2, 3, 4, 5\}, \\ [2, 5) &= \{2, 3, 4\},\end{aligned}$$

です。

整数区間について、**閉区間表記**に加えて**半开区間表記**も頻繁に用います。

- 空集合を自然に $[i, i)$ と表せる。
- $[l, r]$ の要素数は $r - l + 1$, $[l, r)$ の要素数は $r - l$ である。
- $l < m < r$ のとき, $[l, r] = [l, m - 1] \amalg [m, r]$, $[l, r) = [l, m) \amalg [m, r)$ である。

など、要素数の議論や区間を分割する議論において、記述が簡潔になりやすいためです。

6.3. 部分列，部分配列（連続部分列），部分文字列

列 $A = (A_0, A_1, \dots, A_{n-1})$ の**部分列** (subsequence) とは、いくつかの添字 $i_0, i_1, \dots, i_{k-1}$ ($0 \leq i_0 < i_1 < \dots < i_{k-1} < n$) の部分を順序を保って取り出して得られる列 $(A_{i_0}, A_{i_1}, \dots, A_{i_{k-1}})$ のことです。

A の部分列は、添字の選び方の違いを区別した場合には、空列（長さ 0 の列）も含めて 2^n 個あります。

列 $A = (A_0, A_1, \dots, A_{n-1})$ の**部分配列** (subarray) とは、 $0 \leq l \leq r \leq n$ に対して、半開区間 $[l, r)$ に対応する添字を取り出して得られる列 $(A_l, A_{l+1}, \dots, A_{r-1})$ のことです。部分列との区別を強調して、**連続部分列** (contiguous subsequence) と呼ばれることもあります。

A の部分配列は、添字の選び方の違いを区別した場合には、空列（長さ 0 の列）も含めて $\frac{(n+1)(n+2)}{2}$ 個、空列を除いて $\frac{n(n+1)}{2}$ 個あります。

部分配列と同様に、文字列 $S = S_0 S_1 \dots S_{n-1}$ に対しては、 $0 \leq l \leq r \leq n$ について、半開区間 $[l, r)$ に対応する添字を取り出して得られる文字列を**部分文字列** (substring) といいます。

特に部分配列や部分文字列について、空列を含めて考えるかは、文脈によります。AtCoder Algorithm Lectures では、区別が重要な場合にはなるべく明示する予定です。

日本の競技プログラミングコミュニティでは、部分文字列を部分列の意味で用いる文献も少数ありますが、テクニカルタームとしては標準的ではありません。

6.4. 接頭辞，接尾辞

列 $A = (A_0, A_1, \dots, A_{n-1})$ の半開区間 $[l, r)$ に対応する部分配列のうちで、 $l = 0$ の場合にあたるものを A の**接頭辞** (prefix) といいます。

接頭辞は長さ $0, 1, \dots, n$ に対応する $n + 1$ 個あります（空列を含めず n 個とする場合もあります）。

列 $A = (A_0, A_1, \dots, A_{n-1})$ の半開区間 $[l, r)$ に対応する部分配列のうちで、 $r = n$ の場合にあたるものを A の**接尾辞** (suffix) といいます。

接尾辞は長さ $0, 1, \dots, n$ に対応する $n + 1$ 個あります（空列を含めず n 個とする場合もあります）。

文字列についても同様に接頭辞，接尾辞を定義します。

6.5. 辞書式順序

2 つの列 $A = (A_0, A_1, \dots, A_{n-1})$, $B = (B_0, B_1, \dots, B_{m-1})$ について，これらの間の順序（ $A < B$ であるか， $A = B$ であるか， $A > B$ であるか）を次のように定義します。

- $0 \leq i < \min(|A|, |B|)$ かつ $A_i \neq B_i$ となる i が存在する場合： i をそのようなもののうち最小のものとするとき， $A_i < B_i$ であるか $A_i > B_i$ であるかに応じて $A < B$, $A > B$ を定義する。
- そのような i が存在しない場合： $|A| < |B|$, $|A| = |B|$, $|A| > |B|$ のどれであるかに応じて $A < B$, $A = B$, $A > B$ を定義する。 $A < B$ であるときには A は B の接頭辞， $A > B$ であるときには B は A の接頭辞である。

このようにして定まる $<$, $=$, $>$ の関係を，列の**辞書式順序**（lexicographical order）といいます。

辞書式順序について例えば次が成り立ちます。

$$\text{空列} < (1) < (1, 1, 1) < (1, 2) < (2) < (2, 1).$$

文字列についても同様に辞書式順序を定めます。なおその名の示唆する通り，通常の辞書では単語が文字列の辞書式順序によって収録されています。

6.6. 単調性

列 $A = (A_0, A_1, \dots, A_{n-1})$ について，

- $i < j \implies A_i \leq A_j$ が成り立つとき A は**広義単調増加**（weakly increasing）といいます。
- $i < j \implies A_i < A_j$ が成り立つとき A は**狭義単調増加**（strictly increasing）といいます。

広義単調増加であることを，**単調増加**（monotonically increasing）や**非減少**（non-decreasing）ともいいます。ただし，狭義単調増加の意味で「単調増加」という用語を使う文献もあります。

AtCoder Algorithm Lectures では，「**広義単調増加**」「**狭義単調増加**」を用います。単調減少についても同様です。

7. その他の用語

7.1. 定理，命題，系，補題

真（true）であるか偽（false）であるかが定まる文章や式を**命題**（proposition）といいます。ただし，数学書や，AtCoder Algorithm Lectures の解説で命題という見出しで書かれているものは，そのうちでも真であることが証明されたものを指します。

定理（theorem），**系**（corollary），**補題**（lemma）もおおよそ同じ意味で用いられます。ただし

- 定理：命題のうち特に重要なもの。
- 系：他の定理や命題から副産物的に得られるもの。
- 補題：他の定理や命題を証明するための補助的な命題。

というニュアンスがあります。これらの用語には使い分けの客観的な基準はなく，筆者によって主観的に使い分けられます。

7.2. 定義

用語や記号の意味を定めたもの（一定の文献の中で約束したもの）を**定義**（definition）といいます。本講座で扱った内容の大部分は定義であって，命題や定理ではありません。

8. 関連問題

特にありません。

9. まとめ

本講座では、競技プログラミングの問題文や解説，AtCoder Algorithm Lectures で頻出の用語や記号のうち，特に多くの分野に共通するものを簡単にまとめました。

本講座が，多くの読者にとって，競技プログラミングや，数学，情報科学などの諸分野により円滑に入門する助けとなることを期待しています。

10. 参考文献

特にありません。

トップページ：[AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など：[Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

各サービス

[AtCoderとは？](#)

[AtCoder](#) 

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)